

PromptZal

Manual técnico — Proyecto 1

Estructura, análisis léxico, AFD, gramática, errores, reportes y Graphviz

Segundo semestre 2026

Autor: Jonathan Zorin

Contenido

Contenido	2
1. Propósito y alcance	3
2. Organización del proyecto	3
3. Recorrido de la entrada	3
4. Categorías de tokens	3
5. Reconocimiento en Lexer	4
Identificadores y palabras	4
Números	4
Cadenas y directivas	4
Comentarios	4
6. AFD	4
7. Gramática regular	5
8. Recuperación de errores	6
9. Interfaz gráfica	6
10. Reportes HTML	7
11. Graphviz	7
12. Pruebas	8
13. Qué conviene saber explicar	8

1. Propósito y alcance

Este manual explica la parte técnica de PromptZal Proyecto 1. La aplicación recibe un archivo .pz, realiza el análisis léxico y presenta los tokens y errores encontrados. En esta etapa no se construye el prompt final ni se valida la sintaxis completa.

El Proyecto 1 parte del trabajo de la Práctica 1 y agrega AFD, gramática regular, recuperación de errores, interfaz gráfica, reportes HTML y generación del AFD con Graphviz. [filecite]turn10file2[turn10file4]

2. Organización del proyecto

Clase	Responsabilidad
Main	Inicia la aplicación Swing.
Interfaz_Grafica	Editor, botones, tablas, contadores, menús y acciones.
Lexer	Recorre la entrada y reconoce tokens.
Token	Guarda número, lexema, tipo, fila y columna.
TipoToken	Define las categorías léxicas.
ErrorLexico	Guarda información y ubicación del error.
Estadisticas	Calcula datos del reporte estadístico.
ReportesHtml	Genera los reportes HTML.
GeneradorAFD	Genera DOT y solicita a Graphviz PNG/SVG.

3. Recorrido de la entrada

Lexer mantiene la posición, fila, columna y número del siguiente token. Al iniciar un análisis reinicia esos valores y comienza desde el primer carácter.

```
Entrada .pz
|
v
Lexer.analizar()
|
+-- espacio -----> avanzar
+-- / -----> comentario
+-- " -----> cadena
+-- @ -----> directiva
+-- letra o _ -----> identificador / palabra
+-- dígito -----> entero / decimal
+-- -> -----> conector
+-- = o + -----> operador
+-- { } ( ) , -----> delimitador
+-- otro -----> error y continuar
|
v
tokens + errores
```

Al consumir un salto de línea aumenta la fila y la columna vuelve a 1.

4. Categorías de tokens

Categoría	Elementos	Ejemplo
Directiva	@modelo, @rol, @formato	@modelo

Categoría	Elementos	Ejemplo
Palabra reservada	AGENTE, contexto, variable, EJECUTAR, EXPORTAR	AGENTE
Comando de IA	PREGUNTAR, GENERAR, RESUMIR, ANALIZAR, TRADUCIR, CLASIFICAR, EXTRAER	RESUMIR
Conector	SOBRE, DESDE, EN, COMO, ->	SOBRE
Identificador	letra o _ seguida de letras, dígitos o _	ventas
Cadena	texto entre comillas dobles	"ventas.csv"
Entero	número sin punto	100
Decimal	número con punto	0.7
Operador	= y +	=
Delimitador	{ } () ,	{
Función	CARGAR	CARGAR
EOF	marca interna de fin; no se muestra	—

Estas categorías corresponden a las indicadas en la especificación de PromptZal.
filecite turn10file0 turn10file8

5. Reconocimiento en Lexer

Identificadores y palabras

Si el primer carácter es una letra o _, se acumulan letras, dígitos y _. Al terminar se consulta el mapa PALABRAS. Si el texto está definido, se asigna su tipo; de lo contrario queda como IDENTIFICADOR.

Números

Se acumulan los dígitos. Si aparece un punto, el analizador exige al menos un dígito después del punto y produce DECIMAL. Si falta ese dígito registra NUMERO_INVALIDO.

Cadenas y directivas

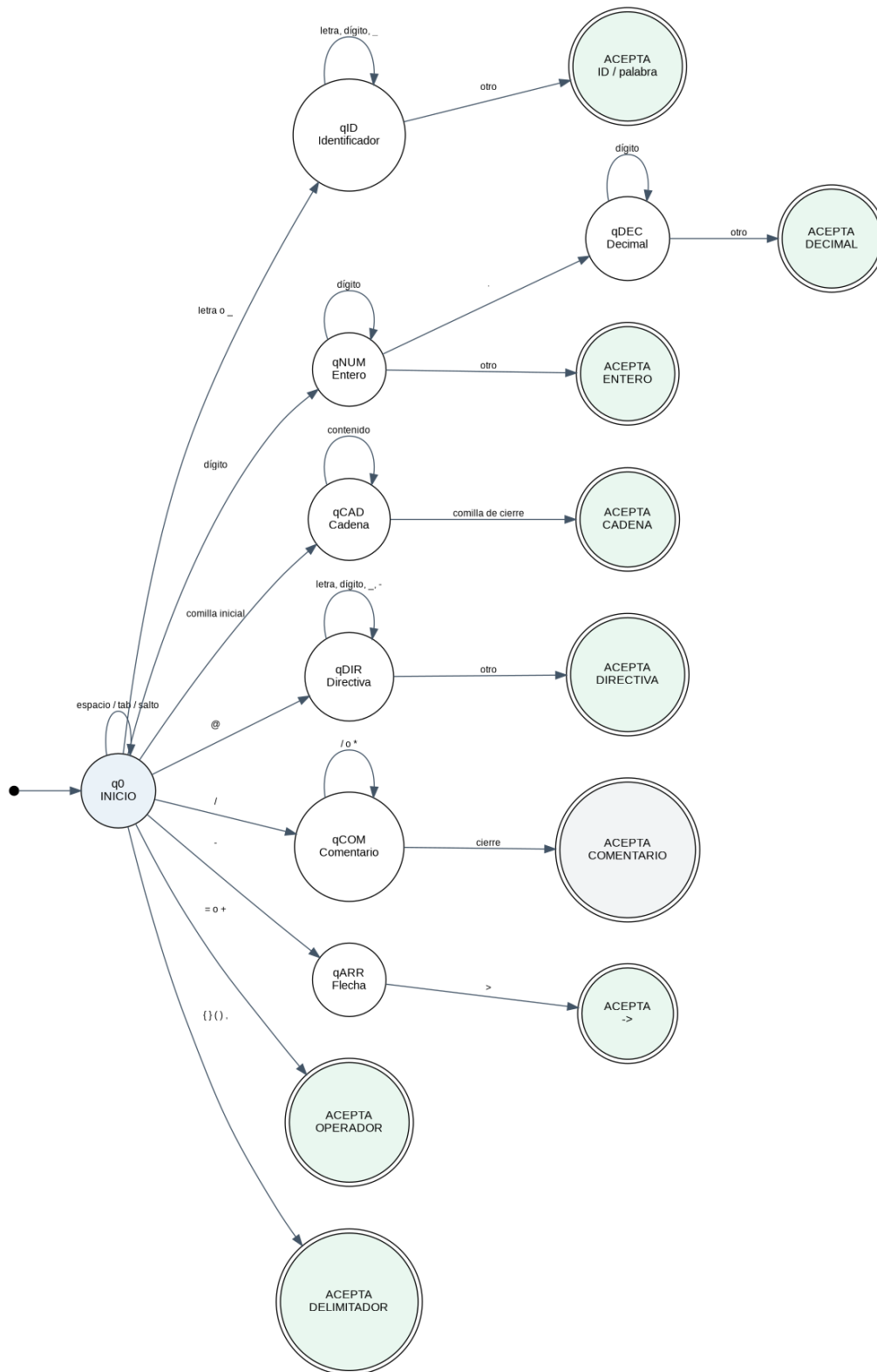
Una cadena comienza con comilla doble y termina en otra comilla. Si no aparece el cierre se registra CADENA_SIN_CERRAR. Una directiva comienza con @ y se compara con @modelo, @rol y @formato; las demás producen DIRECTIVA_INVALIDA.

Comentarios

Los comentarios // se ignoran hasta el final de la línea. Los comentarios /* */ se ignoran hasta encontrar el cierre. Si el cierre no aparece se registra COMENTARIO_SIN_CERRAR.

6. AFD

q0 es el estado inicial. Desde él se selecciona una ruta según el carácter inicial del lexema. Los estados con doble círculo son estados de aceptación.



La imagen corresponde a reportes/afd_promptzal.png. El programa genera el archivo DOT y Graphviz se encarga únicamente de dibujar el grafo; el reconocimiento sigue estando en Lexer. `filecite{turn10file4}`

7. Gramática regular

La gramática se mantiene en un nivel sencillo para relacionarla directamente con el AFD y con los procedimientos de Lexer.

```
ID      -> LETRA ID | _ ID | LETRA | _
NUM     -> DIG NUM | DIG
DEC     -> NUM . DIG DEC | NUM . DIG
CAD     -> " CONTENIDO "
DIR     -> @ NOMBRE
ARROW   -> - >
OP      -> = | +
DEL     -> { | } | ( | ) | ,

LETRA   -> a | b | ... | z | A | B | ... | Z
DIG     -> 0 | 1 | ... | 9
```

Las palabras concretas de PromptZal reutilizan la forma de identificador y luego se clasifican mediante PALABRAS. La validación de la estructura completa del programa no pertenece a este proyecto.

[filecite turn10file8](#)

8. Recuperación de errores

Ante un carácter no reconocido, el Lexer registra el error y avanza una posición. Esto permite continuar y buscar otros errores en la misma entrada. Cada error guarda lexema o carácter, tipo, descripción, fila y columna.

Error	Acción
CARACTER_NO_RECONOCIDO	Registrar y avanzar un carácter.
DIRECTIVA_INVALIDA	Registrar la directiva completa.
CADENA_SIN_CERRAR	Registrar la cadena que quedó abierta.
NUMERO_INVALIDO	Registrar el número que terminó con punto sin dígito.
COMENTARIO_SIN_CERRAR	Registrar el inicio del comentario de bloque.

La especificación pide que el análisis continúe para producir el listado completo de errores.

[filecite turn10file4](#)

9. Interfaz gráfica

La interfaz está hecha con Swing y permite editar, abrir, guardar y analizar archivos .pz, además de acceder a los reportes y al AFD. [filecite turn10file4](#)

Archivos

Reportes HTML

PromptZal

Analizador léxico • Proyecto 1 • Java 21

Analizar

Generar AFD

Abrir

Guardar

Guardar como

Cargar ejemplo

Limpiar

Copiar

```

@modelo "claude-sonnet-4-6"
@rol "analista de datos"
@formato "markdown"

// Agente que prepara el analisis
AGENTE analista {
    contexto = "Eres un analista de datos experto"
    variable ventas = CARGAR("ventas.csv")
    PREGUNTAR "Cuales son las 3 tendencias principales?" SOBRE ventas -> tendencias
    RESUMIR tendencias EN 100 palabras -> resumen
}
EJECUTAR analista
EXPORTAR resumen

```

RESULTADO

ANÁLISIS

Tokens

Errores

TOKENS RECONOCIDOS

37

ERRORES LÉXICOS

0

LÍNEAS

14

Análisis terminado • 37 tokens • 0 errores.

Marca	Función
1	Menús Archivo y Reportes HTML.
2	Analizar y Generar AFD.
3	Abrir, guardar, guardar como, ejemplo, limpiar y copiar.
4	Editor del programa .pz.
5	Pestañas Tokens y Errores.
6	Contadores y estado del último análisis.

10. Reportes HTML

Al presionar Analizar se generan reporte_tokens.html, reporteErrores.html y reporte_estadisticas.html. El reporte de tokens contiene número, lexema, tipo, fila y columna; el de errores muestra el problema y su ubicación; el estadístico presenta frecuencias, totales y líneas.

```
reportes/
reporte_tokens.html
reporteErrores.html
reporte_estadisticas.html
afd_promptzal.dot
afd_promptzal.png
afd_promptzal.svg
```

11. Graphviz

GeneradorAFD crea el DOT y busca el ejecutable dot en el PATH y en rutas habituales de Linux. Si lo encuentra, genera PNG y SVG. Si no lo encuentra, el DOT se conserva y la aplicación informa cómo instalar Graphviz.

```
sudo apt update
sudo apt install graphviz
```

12. Pruebas

Archivo	Tipo	Objetivo
prueba1.pz	Válido	Directivas, agente, variable, CARGAR y comandos.
prueba2.pz	Válido	Cadena, TRADUCIR y EN.
prueba3.pz	Válido	CLASIFICAR, RESUMIR y números.
error1.pz	Con error	Cadena sin cerrar.
error2.pz	Con error	Carácter no reconocido y directiva inválida.

La entrega exige como mínimo cinco casos: tres válidos y dos con errores. [filecite]turn10file5[

13. Qué conviene saber explicar

Conviene dominar el flujo editor → Lexer → tokens/errores → reportes y AFD. También es importante poder explicar token, lexema, fila, columna, identificador, decimal, cadena sin cerrar y recuperación de errores. El documento indica que habrá comprobación de conocimiento y una modificación menor en vivo. [filecite]turn10file5[